

omp parallel region. Threads that complete their jobs early can proceed straight to the next statements in work-sharing constructs, without waiting for other threads to finish. This is typically used if more than one *pragma for* loop is present in the parallel region. The combination of the *schedule* clause (work-load distribution) and the *nowait* clause must be adjusted to optimise the code. The *schedule* clause divides iteration of *pragma for* loops into contiguous subsets known as chunks, and distributes them on threads as per the types. These options can be hard-coded, or can be specified in environment settings at run time. Let's compare two types of *schedule* clauses, *static* and *dynamic*:

- **Static:** Iterations are equally divided and statically distributed among threads. Chunks are calculated automatically, and assigned to threads in a round-robin fashion, depending on the order of the thread number.
- **Static, *chunk_size*:** Iterations are divided into the *N* chunks specified, and statically distributed among threads. The distribution remains the same as in static mode, except that the *chunk size* is also specified here.
- **Dynamic:** Iterations are divided according to the default *chunk size* (1), and dynamically distributed among threads. Iteration chunks are distributed to threads as they request for them. Threads process a chunk, and then request for a new chunk, until no chunks are remaining. The default *chunk_size* is 1, so each thread will be given one iteration.
- **Dynamic, *chunk_size*:** Iterations are divided into the *N* chunks specified, and dynamically distributed among threads. The distribution policy remains the same as in dynamic mode, with the only difference being that each thread will be given *N* iterations to process. Efficient load balancing is achieved using dynamic mode, which also adds processing overhead.



Note: There are more types in the *schedule* clause (such as *guided*, *auto* and *runtime*) which have not been covered in this article. Refer to the OpenMP API guide for details. Depending on your code, I suggest you use the Gprof profiler to check for unexpected wait states in parallel code.

A performance graph of threads vs chunk size

As you can see in Figure 3, a single-thread process results in a different execution time for different chunk sizes. It is evident that one thread on a single core will compete for CPU time, hence one time slot performs operations on *N* loops. A higher chunk size indicates more performance on a single thread, with less overhead of dynamic *schedule* allocation. The single-thread operation of OpenMP code is slower than plain serial code, due to the overheads involved.

For two-thread operations, a chunk size of 2 takes more time to complete than all other chunk sizes. Chunk

size 1 provides maximum performance. For four-thread operations, chunk size 4 yields maximum performance with approximately 188 chunks to be processed by each thread. If we go on increasing the number of threads to the maximum number of cores present in the system, i.e., 128, the performance difference between different chunk sizes vanishes. There is a slight difference between chunk size performances with 128 cores; this is because of the overheads of the dynamic *schedule* option that requests for chunks to process. In dynamic mode, threads are spawned slowly as each thread requests for chunks of data to process. Since there is a possibility of a job being completed before spawning of all specified threads, it is necessary to choose the *chunk size* and number of threads wisely if the system is used for multiple parallel programs.

A performance graph: dynamic vs static schedules

This benchmark is performed with no *chunk size* specified to the *schedule* clause. For the static type of *schedule*, the *chunk size* will be calculated automatically with equal distribution in mind. For the dynamic type, if the *chunk size* is not specified, it is one—so each thread will be provided with one iteration of the *for* loop to process, and after completion it can request a new iteration to process. In NUMA systems, iterations processed closer to the allocated memory location will complete faster, as compared to distant iteration processing nodes, due to memory access latencies. The static *schedule* will load all specified threads immediately with calculated *chunk sizes*. Up to 16 threads, there is a significant difference between the performance of dynamic and static *schedules*, but after that the difference vanishes. The use of dynamic *scheduling* increases performance for a lower number of threads.

- OpenMP behaviour is dynamic; you must tune parameters according to the underlying system to get the best performance.
- Idling threads are of no use at all; try to load threads equally, so that they complete their job at the same time.
- Use OpenMP only when there is sufficient workload.
- Avoid forking/joining of threads at every parallel construct; reuse invoked threads.
- Play with code and observe the performance to obtain the best results.
- Parallel code is very hard to debug, as it silently produces wrong results. Verification is mandatory for error-free parallel code. 

By: Subodh V Pachghare

The author works with India's largest commercial automotive organisation in the IT division. He is interested in the networking, information security, back-up and virtualisation domains. He is a FOSS follower and spends most of his time learning the internals of Linux. You can reach him at www.thesubodh.com or email him at subodhpachghare@gmail.com.